

SemantikML

Improvement Plan: Embeddings & RAG Pipeline

Session 1 implemented · Sessions 2–3 are the roadmap

Context

SemantikML is a txtAI-based ML paper search engine indexing **9,000 documents** across three domains (research/arxiv, blog, documentation). The system uses **all-MiniLM-L6-v2** (384-dim, 256-token max) with FAISS IVF and hybrid BM25+semantic scoring. RAG runs **Qwen/Qwen3-0.6B** locally. This plan reviews improvement options across embeddings, retrieval, and the RAG pipeline, ordered by implementation effort.

Critical Files

File	Purpose
backend/src/search_engine.py	RAG template, context construction, decoding params, cross-encoder
backend/src/build_embeddings.py	Index builder — must re-run for any model or config change
backend/streamlit_app/app_core.py	RAG top_k cap (line 166), SIMILAR_CANDIDATE_BUDGET=25
backend/embeddings/config.json	Live index: IVF230,Flat, 384-dim, 9000 docs, txtai 9.4.1

A. Embedding Model Options

The current model silently truncates documents beyond **256 tokens** (~180 words) — a real problem for arxiv abstracts. Any model swap requires a full re-index.

Model	Dims	Max Tokens	Speed	MTEB	Reindex
all-MiniLM-L6-v2 (current)	384	256	Fastest	~56	No
all-MiniLM-L12-v2	384	256	Fast	~59	Yes
all-mpnet-base-v2	768	384	Moderate	~63	Yes
BAAI/bge-small-en-v1.5 (recommended)	384	512	Fast	~62	Yes
bge-base-en-v1.5	768	512	Moderate	~64	Yes
nomic-embed-text-v1.5	768	8192	Slow	~63	Yes

Recommendation: bge-small-en-v1.5 — retrieval-optimised, same 384-dim (no FAISS config change needed), 512-token window covers most abstracts fully. Files: `build_embeddings.py` and `search_engine.py`.

B. Embedding / Retrieval Improvements

B1 Title + Text Concatenation [Medium — reindex required]

Only text is embedded; title is stored metadata only. Indexing `title [SEP] text` injects title signal into every vector, improving retrieval for title-relevant queries.

- File: `build_embeddings.py` — record tuple construction

B2 Hybrid Weight Tuning [Medium — reindex required]

Hybrid weights are currently implicit. Adding `"weights": {"hybrid": 0.7}` lets you bias toward semantic (conceptual) or BM25 (exact-term) scoring.

- File: `build_embeddings.py` — scoring dict

B3 Cross-Encoder Re-ranking on Main Search [Quick win — no reindex]

The cross-encoder/`ms-marco-MiniLM-L-6-v2` is already loaded for `more_like_this`. Applying it to `search()` (retrieve 20 candidates, re-rank all 20, return top_k) improves `MRR@10` by 5–15%. Adds ~1–3 s latency on CPU.

- File: `search_engine.py` — `search()` function

B4 Document Chunking [Long-term]

Splitting long docs into overlapping passages dramatically improves retrieval for blog/documentation domains. Architecturally disruptive: breaks the 1-to-1 row-to-document assumption across the codebase.

C. RAG Template Improvements (Quick wins — restart only)

All changes require only editing the template string in `search_engine.py` and restarting the server (@st.cache_resource must flush). **C1–C4 implemented in Session 1.**

ID	Change	Action	Impact
C1	Remove third-person constraint	Replace with 'clear, professional tone'	More natural output (done)
C2	Add fallback instruction	State clearly if context is insufficient	Reduces hallucination (done)
C3	Include titles in context	Prefix each doc with Title: ...	Enables citations (done)
C4	Add citation instruction	Reference source titles in parentheses	User trust (done)
C5	Domain context (optional)	Format template dynamically with domain param	Domain-appropriate tone

D. RAG Pipeline Parameter Improvements

D1 max_context_chars: 1500 → 3000 [Quick win — done]

Qwen3-0.6B has a 32k-token context window — the limit was in the code, not the model. Doubling this feeds more of each retrieved document to the LLM.

D2 Sampling Decoding [Quick win — done]

Replaced greedy `do_sample=False` with `do_sample=True`, `temperature=0.7`, `top_p=0.8` (Qwen3 model card recommendations). Eliminates repetitive output loops.

D3 Raise RAG top_k cap: 5 → 7 [Quick win — done]

Raised the cap in `app_core.py` so users selecting higher top-k values actually see more context fed to the LLM.

D4 LLM Upgrade [Medium effort]

Model	VRAM (FP16)	Speed	Notes
Qwen3-0.6B (current)	~1.2 GB	Fast	Baseline
Qwen3-1.7B (recommended)	~3.4 GB	Moderate	3x better instruction following; 1-line change
Qwen3-4B	~7 GB	Slow on CPU	Needs CUDA or quantisation
Cloud API (OpenAI/Anthropic)	N/A (pay/token)	Fast (network)	Production quality; txtai native support

Recommended Implementation Order

D1: Increase max_context_chars 1500 → 3000	Quick win
D2: Sampling decoding (temp=0.7, top_p=0.8)	Quick win
C1–C4: Template — tone, fallback, titles, citations	Quick win
D3: Raise RAG top_k cap 5 → 7	Quick win
Swap to bge-small-en-v1.5 + title+text concatenation	Medium
Add cross-encoder re-ranking to search() (B3)	Quick win
Upgrade to Qwen3-1.7B if memory allows (D4)	Medium
Document chunking for blog/documentation domains (B4)	Long-term
Cloud LLM for public-facing deployment (D4)	Long-term

Verification

- After template/param changes: restart Streamlit, run RAG queries across each domain; verify answers are longer, more natural, cite titles, and admit uncertainty when context is insufficient.
- After reindex: compare top-5 results for 5 representative queries before/after; confirm longer abstracts are no longer truncated mid-sentence.
- Cross-encoder re-ranking (B3): verify total search latency stays below 5 s on CPU.
- LLM upgrade (D4): run 10 identical RAG queries with 0.6B and 1.7B; compare citation adherence and fallback behaviour.